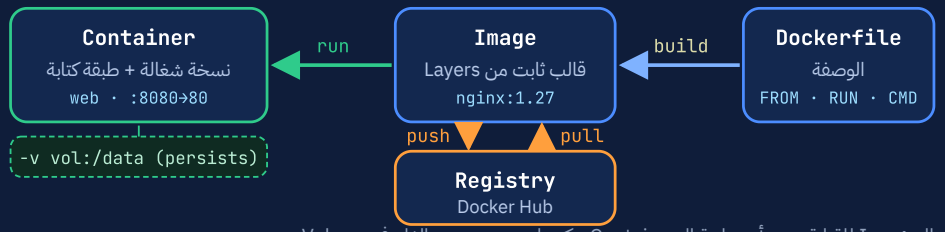
الـ Images — Images — الفصل 3

- ف3 ★ هات Image بنسخة محددة: `docker pull nginx:1.27`
- ف3 ★ اللي عندك: `docker images`
- ف3 امسح Image: `docker rmi nginx:1.27`
- ف6 الـ Layers: `docker image history nginx`
- ف3 امسح اللي ملهاش اسم: `docker image prune`

من الوصفة للـ Container — Dockerfile → Image → Container — ف2 · ف6



الـ Image للقراية بس. أي حاجة الـ Container بيكتبها بتروح مع rm، إلا لو في Volume.

التشغيل — Run — الفصل 4

- ف4 ★ في الخلفية + بورت: `docker run -d -p 8080:80 nginx`
- ف4 ★ تفاعلي ويتمسح لما تخرج: `docker run --rm -it ubuntu bash`
- ف4 Environment variable: `docker run -e PORT=9090 app`
- ف4 بورت عشوائي لكل EXPOSE: `docker run -d -P nginx`

إدارة الـ Containers — Containers — الفصل 4

- ف4 ★ الشغالين: `docker ps`
- ف4 ★ كلهم حتى الواقفين: `docker ps -a`
- ف4 ★ ادخل جوه Container شغال: `docker exec -it web sh`
- ف4 وقّف · شغّل: `docker stop web · docker start web`
- ف4 امسح Container واقف: `docker rm web`
- ف4 ⚠ وقّف وامسح بالإجبار: `docker rm -f web`

الـ Volumes — البيانات اللي تعيش — الفصل 7

- ف7 ★ Named volume: `docker run -v my-vol:/data app`
- ف7 ★ Bind mount: فولدر من جهازك: `docker run -v "$PWD":/app app`
- ف7 Read-only: `docker run -v my-vol:/data:ro app`
- ف7 في الـ RAM بس: `docker run --tmpfs /tmp app`
- ف7 إدارة: `docker volume ls · create · inspect`
- ف7 ⚠ امسح Volume: `docker volume rm my-vol`

dockerignore. — Build context — الفصل 11

```
node_modules
.git
dist
*.log
.env
```

من غيره كل الفولدر بيتبعت للـ build: بطء، وأسرار ممكن تدخل الـ Image.

البناء والرفع — Build & Push — ف10 · ف12

- ف10 ★ ابني من الـ Dockerfile هنا: `docker build -t app .`
- ف10 من ملف باسم تاني: `docker build -t app -f Dev.Dockerfile .`
- ف9 قيمة لـ ARG: `docker build --build-arg V=2 -t app .`
- ف12 ★ اسم بحسابك قبل الرفع: `docker tag app sara/app:1.0`
- ف12 ادخل Docker Hub: `docker login`
- ف12 ★ ارفع: `docker push sara/app:1.0`

تشريح أمر: docker run — ف4

```
1 docker  2 run  3 -d  4 -p  5 8080:80
6 --name web  7 nginx:1.27
```

1. الأداة
2. اعمل Container وشغّله
3. في الخلفية
4. افتح بورت
5. Host:Container
6. اسم بدل ID
7. Image:Tag

الـ Dockerfile — Multi-stage — ف9 · ف10

```
# 1) build stage
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# 2) runtime stage
FROM nginx:1.27-alpine
WORKDIR /usr/share/nginx/html
COPY --from=builder /app/dist .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- ف10 ★ الـ Dependencies الأول = Cache أحسن: `COPY package*.json ./` → `RUN npm ci`
- ف9 متغير جوه الـ Container: `ENV PORT=8080`
- ف9 وقت الـ build بس: `ARG VERSION=1.0`
- ف9 ★ متشغّلش كـ root: `USER appuser`
- ف9 ثابت + افتراضي: `ENTRYPOINT ["ping"]` + `CMD ["x.com"]`
- ف9 Metadata: `LABEL author="sara"`
- ف9 Anonymous volume: `VOLUME /data`
- ف9 نسخ مع فك ضغط: `ADD app.tar.gz /app/`

المراقبة — Monitoring — الفصل 5

- ف5 ★ تابع آخر 50 سطر: `docker logs -f --tail 50 web`
- ف5 كل التفاصيل JSON: `docker inspect web`
- ف5 قيمة واحدة: `docker inspect -f '{{.State.Status}}' web`
- ف5 RAM و CPU: `docker stats --no-stream`
- ف5 المساحة: `docker system df`

Docker Compose
الفصل 13 · compose.yaml

```yaml
services:
  backend:
    build: ./backend
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgres://db:5432/app
    depends_on: [db]
  db:
    image: postgres:17
    volumes: [db-data:/var/lib/postgresql/data]
volumes:
  db-data:
```

- ★ ف13 شغّل كل الـ services
  `docker compose up -d`
- ★ ف13 ابني تاني وشغّل
  `docker compose up -d --build`
- ف13 الحالة — `docker compose ps`
- ف13 لوج service
  `docker compose logs -f backend`
- ف13 ادخل جوه service
  `docker compose exec backend sh`
- ف13 وقّف وامسح (الـ Volumes باقية)
  `docker compose down`
- ⚠ ف13 ومعاهم الـ Volumes
  `docker compose down -v`

الشبكات Networks
الفصل 8

- ★ ف8 شبكة فيها DNS
  `docker network create app-net`
- ★ ف8 شغّل جوه الشبكة
  `docker run --network app-net --name db postgres`
- ف8 من Container تاني: بالاسم — `ping db`
- ف8 وصّل واحد شغال
  `docker network connect app-net web`
- ف8 اعرض — `docker network ls · inspect`
- ف8 شبكة الجهاز نفسه
  `docker run --network host nginx`

الـ Layers Image structure
الفصل 6

- ف6 كل Layer وحجمها — `docker image history app`
- ف9 في RUN واحد = Layer أقل
  `RUN apt-get update && apt-get install -y x`
- ★ ف9 صغير = Image أصغر Base
  `FROM node:22-alpine`

الـ Image Layers للقراية بس. الـ Container بيكتب في طبقة فوقها (Copy-on-Write)، وبتتمسح مع الـ Container.

التنضيف (بيمسح نهائي) Prune
الفصل 5

- ⚠ ف4 كل الواقفين — `docker container prune`
- ⚠ ف3 كل Image مش مستخدمة
  `docker image prune -a`
- ⚠ ف7 Volumes مش مستخدمة = بيانات!
  `docker volume prune`
- ⚠ ف5 كله مرة واحدة — `docker system prune -a`

أوامر مساعدة Helpers
الفصل 5

- ف5 الـ Processes جوه الـ Container — `docker top web`
- ف5 انسخ ملف من الـ Container
  `docker cp web:/app/log.txt .`
- ف2 Client و Engine شغالين؟
  `docker version · docker info`
- ف5 عرض مختصر بالـ Template
  `docker container ls --format "{{.Names}}"`

تانية Registries
الفصل 12

- ف12 GitHub Container Registry
  `ghcr.io/user/app:1.0`
- ف12 الاسم الكامل لـ nginx
  `docker.io/library/nginx:latest`

حاجات بتتلخبط بينها X vs Y
قارن بسرعة

بدل	الفرق	خلي بالك	ف
المفاهيم			
Image / Container	القالب الثابت / نسخة شغالة منه	Image واحدة ← Containers كتير	ف2
VM / Container	نظام تشغيل كامل / بيشارك الـ Kernel	الـ Container أخف وأسرع	ف1
الـ Dockerfile			
`ENTRYPOINT` / `CMD`	افتراضي يتغير / البرنامج الثابت	مع بعض: ENTRYPOINT + CMD args	ف9
`CMD` / `RUN`	وقت الـ build / وقت التشغيل		ف9
`ADD` / `COPY`	نسخ بس / نسخ + فك ضغط	استخدم COPY	ف9
`ARG` / `ENV`	جوه الـ Container / وقت الـ build بس	متحطش أسرار في الاتنين	ف9
`-p` / `EXPOSE`	يافطة توثيق / بيفتح البورت فعلًا		ف9
البيانات والشبكات			
Named volume / Bind mount / tmpfs	Docker بيديره / فولدر من جهازك / RAM	Bind للتطوير · Named لـ DB	ف7
default bridge / user network	من غير DNS / بالأسماء	اعمل network لمشروعك	ف8
`down -v` / `down`	الـ Volumes باقية / بتتمسح		ف13

لو حصل كذا... اعمل كذا Troubleshooting
مشاكل شائعة

ابدأ بـ `docker ps -a` و `docker logs web` : الـ Container اللي وقف لسه موجود ولوجه معاه.

- ف2 **Cannot connect to the Docker daemon** · Docker Desktop مش شغال
  `open -a Docker`
- ف4 **الـ Container اشتغل وخلص على طول** · مفيش Process شغالة جواه
  `docker logs web` `docker run -it ubuntu bash`
- ف4 **port is already allocated** · البورت مستخدم
  `docker ps` `-p 8081:80`
- ف7 **البيانات ضاعت بعد rm** · مكانش فيه Volume
  `-v my-vol:/data`
- ف8 **Container مش شايف التاني بالاسم** · على الـ default bridge
  `docker network create app-net`
- ف8 **localhost جوه Container مش شغال** · الـ localhost = Container نفسه
  `postgres://db:5432`
- ف11 **الـ build بطيء أو ضخم** · الـ Context فيه كراكيب
  `.dockerignore` `multi-stage`
- ف13 **عدّلت الكود ومتغيرش** · الـ Image القديمة
  `docker compose up -d --build`
- ف12 **push: access denied** · الاسم مش باسم حسابك
  `docker tag app sara/app:1.0` `docker login`
- ف5 **الـ Disk اتملى** · Images و Volumes قديمة
  `docker system df`